

1. Single-select Multiple Choice (10 Questions)

1. Copilot Studio knowledge source

Q: Which service best organizes heterogeneous business data into a searchable knowledge source for Copilot Studio?

- A. Azure AI Search
- B. Azure Data Lake Storage
- C. Azure Cosmos DB
- D. Azure Translator

Answer: A. Azure AI Search — optimized for semantic retrieval.

2. ALM deployment Q: Which tool ensures repeatable deployments of Power Platform solutions?

- A. ALM Accelerator
- B. Azure DevOps Pipelines
- C. GitHub Codespaces
- D. Dynamics 365 Lifecycle Services

Answer: A. ALM Accelerator — purpose-built for Power Platform ALM.

3. Responsible AI principle Q: Which principle directly mitigates bias in AI agents?

- A. Reliability
- B. Fairness
- C. Security
- D. Transparency

Answer: B. Fairness — ensures equitable outcomes.

4. Security design Q: Which mitigation prevents prompt injection attacks?

- A. Role-based access control
- B. Input sanitization
- C. Encryption at rest
- D. Multi-factor authentication

Answer: B. Input sanitization — filters malicious prompts.

5. Dynamics 365 integration

Q: Which service synchronizes Finance and Sales data?

- A. Dataverse
- B. Azure Synapse
- C. Power BI
- D. Azure Functions

Answer: A. Dataverse — common data model.

6. Monitoring telemetry

Q: Which tool provides agent telemetry dashboards?

- A. Azure Monitor
- B. Power BI
- C. Dynamics Lifecycle Services
- D. Microsoft Foundry

Answer: A. Azure Monitor — native telemetry.

7. Multi-agent orchestration

Q: Which protocol enables agent-to-agent communication?

- A. REST
- B. A2A
- C. SOAP
- D. OData

Answer: B. A2A — agent-to-agent orchestration.

8. Cloud migration framework

Q: Which framework guides Azure migration?

- A. Success by Design
- B. Cloud Adoption Framework
- C. Well-Architected Framework
- D. ALM Accelerator

Answer: B. Cloud Adoption Framework — migration methodology.

9. Conversational UX

Q: Which framework ensures conversational design best practices?

- A. Success by Design
- B. Well-Architected Framework
- C. ALM Accelerator
- D. Cloud Adoption Framework

Answer: B. Well-Architected Framework — UX principles.

10. ROI calculation

11. Q: Which tool measures AI agent ROI?

Power BI

Azure Cost Management

Dynamics 365 Sales Insights

Dataverse

Answer: B. Azure Cost Management — cost vs. savings.

2. Multi-select Multiple Choice (10 Questions)

1. **Quarterly agent analysis** Which two metrics should CFO evaluate?

- A. GPT models used
- B. Avg. characters per message
- C. Avg. session time
- D. Orchestration method

Answer: C, D

2. **Security compliance** Which three controls mitigate risks?

- A. Role-based access control
- B. Encryption at rest
- C. Prompt injection sanitization
- D. Session length limits

Answer: A, B, C

3. **Responsible AI adoption** Which two principles apply?

- A. Transparency

- B. Reliability
- C. Cost optimization
- D. Scalability

Answer: A, B

4. **ALM pipeline components** Which three are required?

- A. Source control repo
- B. Automated build pipeline
- C. Manual deployment scripts
- D. Test environment

Answer: A, B, D

5. **Monitoring telemetry** Which two metrics matter most?

- A. Error rate
- B. User satisfaction score
- C. Number of prompts
- D. Average keystrokes

Answer: A, B

6. **Dynamics 365 integration** Which two services enable cross-app workflows?

- A. Dataverse
- B. Power Automate
- C. Azure Functions
- D. Azure Blob Storage

Answer: A, B

7. **Multi-agent orchestration** Which two protocols apply?

- A. A2A
- B. REST
- C. SOAP
- D. SMTP

Answer: A, B

8. **Cloud migration governance** Which two frameworks apply?

- A. Success by Design
- B. Cloud Adoption Framework
- C. ITIL
- D. COBIT

Answer: A, B

9. **Testing types** Which two ensure workflow alignment?

- A. Integration testing
- B. User acceptance testing
- C. Unit testing
- D. Regression testing

Answer: A, B

10. **ROI reporting** Which two tools apply?

- A. Power BI
- B. Azure Cost Management
- C. Dynamics Lifecycle Services
- D. GitHub Actions

Answer: A, B

3. Drag-and-Drop (10 Questions)

Examples (answers shown inline):

1. Agent deployment methods

Copilot Studio → ALM Accelerator

Foundry agent → Azure DevOps Pipelines

2. Testing types

Data exchange → Integration testing

Workflow alignment → User acceptance testing

3. Security controls

Prompt injection → Input sanitization

Unauthorized access → RBAC

4. **Telemetry tools**

Logs → Azure Monitor

Dashboards → Power BI

5. **Migration steps**

Assess → Cloud Adoption Framework

Govern → Success by Design

6. **Orchestration protocols**

Agent-to-agent → A2A

External API → REST

7. **Knowledge source setup**

Unstructured docs → Azure AI Search

Tabular data → Dataverse

8. **Responsible AI principle mapping**

Bias mitigation → Fairness

Explainability → Transparency

9. **ALM pipeline stages**

Build → GitHub Actions

Deploy → ALM Accelerator

10. **Monitoring metrics**

Latency → Azure Monitor

Adoption → Power BI

4. **Drop-down Selection (10 Questions)**

Examples:

1. **Testing type for workflow alignment** → User acceptance testing
2. **Testing type for cross-app data exchange** → Integration testing
3. **Component for ROI reporting** → Power BI
4. **Component for cost tracking** → Azure Cost Management

5. **Security control for prompt injection** → Input sanitization
6. **Framework for migration** → Cloud Adoption Framework
7. **Framework for conversational UX** → Well-Architected Framework
8. **Deployment tool for repeatability** → ALM Accelerator
9. **Knowledge source for Copilot Studio** → Azure AI Search
10. **Telemetry tool for dashboards** → Power BI

5. Case Study (8 Questions across 2 Scenarios)

Case Study 1: JitendraCo migration

- Q1: Which framework for migration? → Cloud Adoption Framework
- Q2: Which tool for ALM? → ALM Accelerator
- Q3: Which principle ensures bias mitigation? → Fairness
- Q4: Which escalation rule applies? → Escalate after two failed attempts

Case Study 2: Dynamics 365 integration

- Q5: Which common data source? → Dataverse
- Q6: Which testing type ensures workflow alignment? → User acceptance testing
- Q7: Which monitoring tool for telemetry? → Azure Monitor
- Q8: Which reporting tool for ROI? → Power BI

6. Yes/No Hot Spot Questions (Scenario: Sales executives use headsets to query AI agent)

1. Should the AI agent escalate after two failed attempts?

Yes — ensures customer support continuity.

2. Should sensitive data be ignored in logs?

No — must be logged for auditing, but protected.

3. Should session telemetry be monitored?

Yes — adoption and performance tracking.

10 Drag-and-Drop (Match technologies / order steps)

1. **Data ingestion pipeline**

- Raw IoT telemetry → Azure Event Hub

- Stream processing → Azure Stream Analytics
- Storage → Azure Data Lake Gen2
- Knowledge source → Azure AI Search

2. **ALM stages for Copilot Studio agent**

- Source control → GitHub
- Build → GitHub Actions
- Test → Sandbox environment
- Deploy → ALM Accelerator

3. **Security controls mapping**

- Prompt injection → Input sanitization
- Unauthorized access → RBAC
- Data exfiltration → DLP policies
- Compliance logging → Azure Monitor

4. **Multi-agent orchestration**

- Agent-to-agent → A2A protocol
- External API → REST
- Legacy ERP → OData
- Messaging → Service Bus

5. **Responsible AI principles**

- Bias mitigation → Fairness
- Explainability → Transparency
- Reliability → Robust testing
- Privacy → Differential privacy

6. **Monitoring telemetry sources**

- Application logs → Azure Monitor
- Business KPIs → Power BI
- User sentiment → Dynamics Customer Voice
- Deployment health → Azure DevOps

7. **Cross-app integration**

- Dynamics 365 Finance ↔ Dynamics 365 Sales → Dataverse
- Workflow automation → Power Automate
- Analytics → Synapse
- Reporting → Power BI

8. **Governance framework steps**

- Assess → Cloud Adoption Framework
- Govern → Success by Design
- Optimize → Well-Architected Framework
- Operate → Azure Monitor

9. **Testing types**

- Workflow alignment → UAT
- Data exchange → Integration testing
- Component validation → Unit testing
- Regression → Automated test suite

10. **Deployment isolation**

- Dev → Sandbox environment
- Test → Staging environment
- Prod → Isolated tenant
- Monitoring → Azure Monitor

10 Dropdown (Architectural diagram completion)

1. Knowledge source for heterogeneous docs → **Azure AI Search**
2. Common data layer for Dynamics apps → **Dataverse**
3. Deployment automation → **ALM Accelerator**
4. Telemetry dashboards → **Power BI**
5. Security for prompt injection → **Input sanitization**
6. Governance framework → **Success by Design**
7. Migration methodology → **Cloud Adoption Framework**

8. Orchestration protocol → **A2A**
9. Cost optimization tool → **Azure Cost Management**
10. Workflow automation → **Power Automate**

Case Studies (8 Questions across 2 scenarios)

Case Study 1: Global Retailer AI Deployment

Scenario: Retailer wants Copilot Studio agents for customer service, integrated with Dynamics 365 Finance and Supply Chain. Data resides in SQL, Cosmos DB, and flat files. Compliance requires GDPR adherence.

- Q1: Which knowledge source? → Azure AI Search
- Q2: Which integration layer? → Dataverse
- Q3: Which testing type ensures Finance ↔ Supply Chain workflows? → Integration testing
- Q4: Which Responsible AI principle ensures GDPR compliance? → Privacy
- Q5: Which deployment tool ensures isolation? → ALM Accelerator
- Q6: Which monitoring tool for telemetry? → Azure Monitor
- Q7: Which reporting tool for ROI? → Power BI
- Q8: Which governance framework ensures compliance? → Success by Design

Case Study 2: Government Financial Program

Scenario: Agency wants Foundry agents for fraud detection, orchestrated with Copilot Studio. Needs multi-agent orchestration, strict compliance, and automated deployments.

- Q1: Which orchestration protocol? → A2A
- Q2: Which security control mitigates prompt injection? → **Input sanitization**
- Q3: Which framework ensures migration governance? → **Cloud Adoption Framework**
- Q4: Which tool for automated deployments? → Azure DevOps Pipelines
- Q5: Which telemetry source for fraud detection KPIs? → **Power BI**
- Q6: Which Responsible AI principle ensures fairness? → Fairness
- Q7: Which testing type validates fraud detection models? → **Regression testing**
- Q8: Which monitoring tool ensures deployment health? → **Azure Monitor**

Yes/No Hot Spot (Scenario: Bank deploys AI agent for loan approvals)

1. Should the AI agent escalate to human review after two failed identity verifications? **Yes** — ensures compliance and reduces fraud risk.
2. Should sensitive customer financial data be excluded from logs? **Yes** — logs must anonymize or mask sensitive data to comply with GDPR/PCI.
3. Should telemetry capture average decision latency per agent? **Yes** — latency impacts customer experience and system scalability.

This set is **architect-level difficulty**: it forces trade-off reasoning across **security, governance, orchestration, compliance, and multi-platform integration**.

Would you like me to extend this into a **full 50-question mock exam** so you can simulate the entire AB-100 experience end-to-end?

.

Section 1 — Single-Select Multiple Choice (10)

Q1. A retailer wants a Copilot Studio agent to answer questions using **unstructured PDF product manuals stored in SharePoint, with minimal setup and no custom indexing pipeline**. What should you configure as the knowledge source?

- A. Azure AI Search index
- B. SharePoint site/folder as a generative answers knowledge source
- C. Azure Cosmos DB with vector search
- D. Dataverse table import

Answer: B — Native SharePoint knowledge sources in Copilot Studio handle document ingestion and grounding automatically;

Azure AI Search (A) is the right call only when you need custom indexing, chunking control, or non-SharePoint structured content.

Q2. You are designing an agent that **must call a legacy on-premises SOAP API not exposed to the internet**. Which component should you use to let the agent invoke this system securely?

- A. A generative AI orchestration prompt

- B. An on-premises data gateway with a custom connector
- C. Azure AI Search connector
- D. A Power Automate cloud-only flow

Answer: B — The on-premises data gateway is the standard bridge for reaching internal systems from cloud services like Copilot Studio/Power Platform; cloud-only flows (D) cannot reach non-internet-exposed endpoints.

Q3. A CFO wants an agent that flags invoices exceeding a variable, business-rule-driven threshold before approval. Which Copilot Studio capability best fits deterministic, rule-based logic rather than generative reasoning?

- A. Generative orchestration with a broad system prompt
- B. Topics with classic (structured) trigger phrases and conditions
- C. A single knowledge-source-only agent
- D. An autonomous agent with no topics

Answer: B — Deterministic business rules map to classic topics/conditions, which are predictable and auditable; generative orchestration (A) is better for open-ended reasoning, not strict rule enforcement.

Q4. You need to ensure an agent's responses to regulated financial questions never hallucinate figures and are always traceable to source documents. What should you enable?

- A. Increase model temperature
- B. Strict grounding to knowledge sources with citations enabled
- C. Disable knowledge sources entirely
- D. Use only generative orchestration with no knowledge sources

Answer: B — Grounding + citations constrains answers to source material and lets reviewers trace outputs; raising temperature (A) increases variability, the opposite of what's needed.

Q5. Which Azure service would you recommend for centrally monitoring token usage, latency, and error rates across multiple Foundry-hosted agents in production?

- A. Azure Monitor / Application Insights
- B. Azure DevOps Boards
- C. Microsoft Purview only
- D. Power BI Desktop

Answer: A — Azure Monitor/App Insights is purpose-built for telemetry (latency, errors, usage) across Azure-hosted workloads including Foundry agents; Purview (C) is for governance/data classification, not runtime telemetry.

Q6. A company wants to prevent an agent from being manipulated by malicious instructions embedded inside a user-uploaded document (prompt injection via content). What is the most effective mitigation to recommend?

- A. Increase the agent's max response length
- B. Content filtering/prompt shields plus instruction-hierarchy separation between system and content
- C. Remove all knowledge sources
- D. Rely solely on end-user training

Answer: B — Prompt shields and clear system/content separation are the recommended technical mitigation for indirect prompt injection; user training (D) helps but isn't a technical control.

Q7. You must choose a deployment approach for a Copilot Studio agent that guarantees isolated dev, test, and prod environments with promotion via solutions. What should you use?

- A. Manual export/import of individual topics
- B. Managed environments with solution-based ALM across environment tiers
- C. A single shared environment for all stages
- D. Direct publishing from a developer's default environment to production

Answer: B — Solution-based ALM across dev/test/prod managed environments is the supported pattern for isolation and repeatable promotion in Power Platform/Copilot Studio.

Q8. Which Responsible AI principle is primarily addressed by requiring human review before an autonomous agent executes an irreversible action (e.g., issuing a refund over \$10,000)?

- A. Transparency
- B. Human oversight / accountability
- C. Fairness
- D. Inclusiveness

Answer: B — Human-in-the-loop checkpoints for high-impact actions map to the accountability/human oversight principle in Microsoft's Responsible AI framework.

Q9. A Dynamics 365 Finance customer wants an in-app Copilot experience that answers "how do I" questions using official Microsoft help content plus company-specific policy documents. What should you recommend?

- A. Build a fully custom agent from scratch in Foundry only
- B. Extend the built-in Dynamics 365 Finance Copilot with additional knowledge sources
- C. Disable the built-in Copilot and use email support
- D. Use Power BI Q&A visuals

Answer: B — Extending the existing in-app Copilot with supplemental knowledge sources is the recommended low-effort, high-fit approach rather than rebuilding from scratch.

Q10. You need to choose between Conversational Language Understanding (CLU), classic topics, and generative orchestration for an agent handling highly varied natural-language requests where intent isn't easily enumerated.

What is the best fit?

- A. Classic topics only
- B. Generative orchestration
- C. CLU with a fixed intent list only
- D. No AI capability, static FAQ

Answer: B — Generative orchestration is designed for open-ended, high-variability intent handling where a fixed set of trigger phrases or intents would be impractical to enumerate.

Section 2 — Multi-Select Multiple Choice (10)

Q11. Which two factors should you evaluate when deciding whether an agent should use generative orchestration versus classic topics? (Choose 2)

-) A. The variability and unpredictability of user phrasing
- B. The color scheme of the Copilot Studio canvas
- C. Whether responses must be deterministic/auditable for compliance
- D. The number of Teams channels the org uses

Answer: A, C

Q12. Which two elements are part of a mature ALM strategy for Copilot Studio and Foundry agents? (Choose 2)

- A. Automated CI/CD pipelines with solution checker/validation gates
- B. Publishing directly to production without a test environment
- C. Version-controlled solutions in a source repository
- D. Allowing all makers to edit production directly

Answer: A, C

Q13. Which two risks should a security review specifically assess for an agent that can call external tools/APIs on a user's behalf? (Choose 2)

- A. Over-permissioned connections that exceed least-privilege
- B. Font rendering issues in the chat widget
- C. Data exfiltration via tool outputs returned to untrusted contexts
- D. The agent's average response character count

Answer: A, C

Q14. Which two metrics are most useful for a quarterly business review of agent effectiveness? (Choose 2)

- A. Escalation/handoff rate to human agents
- B. The exact GPT model version string
- C. Task completion / resolution rate
- D. Number of environment variables defined

Answer: A, C

Q15. When designing a multi-agent orchestration where a "router" agent delegates to specialist agents, which two design considerations matter most? (Choose 2)

- A. Clear, non-overlapping scope/responsibility boundaries per specialist agent
- B. Using the same exact system prompt for every specialist agent
- C. A reliable handoff/context-passing mechanism between agents
- D. Disabling logging on the router agent

Answer: A, C

Q16. Which two capabilities does Microsoft Foundry provide that are relevant to enterprise agent governance? (Choose 2)

- A. Centralized model/connection management and access control
- B. Automatic invoice generation for customers
- C. Content safety and evaluation tooling for deployed models/agents
- D. Built-in HR payroll processing

Answer: A, C

Q17. Which two are valid reasons to use Azure AI Search as a knowledge source instead of a native SharePoint/Dataverse connector? (Choose 2)

- A. You need custom chunking, embeddings, or hybrid semantic + keyword search
- B. The content is a single small FAQ document

- C. You need to index data from multiple heterogeneous sources into one searchable index
- D. You want to avoid any indexing at all

Answer: A, C

Q18. Which two practices support the Responsible AI principle of transparency in an agent solution? (Choose 2)

- A. Disclosing to users that they are interacting with an AI agent
- B. Hiding all citations from end users
- C. Providing citations/sources for generated answers
- D. Suppressing all error messages

Answer: A, C

Q19. Which two considerations should guide your choice between Model Context Protocol (MCP) and Agent2Agent (A2A) for an integration? (Choose 2)

- A. MCP is typically used to connect an agent to tools/data sources
- B. A2A is typically used for agent-to-agent communication and delegation across systems
- C. MCP and A2A are mutually exclusive and can never be used in the same solution
- D. Both protocols eliminate the need for authentication

Answer: A, B

Q20. Which two items should be included in a data loss prevention (DLP) review before deploying an agent broadly? (Choose 2)

- A. Which connectors are classified as business vs. non-business data groups
- B. The agent's avatar image
- C. Whether sensitive data could flow to an ungoverned connector
- D. The number of topics in the agent

Answer: A, C

Section 3 — Drag-and-Drop (Match / Order) (10)

For each, match the item on the left to the correct target on the right (answers given below each question).

Q21. Match the deployment method to the agent platform requirement Items:

1) Copilot Studio agent needing automated, repeatable, no-manual-intervention deployment

2) Foundry agent in a code-first repo needing the same Targets:

A) Solution pipelines (managed environments, ALM CI/CD)

B) Azure DevOps/GitHub Actions pipeline deploying Foundry resources via IaC (Bicep/ARM/Terraform)

Answer: 1→A, 2→B

Q22. Match the testing type to its purpose Items:

1) Integration testing

2) User acceptance testing (UAT) Targets:

A) Validates data exchange correctly flows between Dynamics 365 Sales and Finance

B) Confirms the solution aligns with real business workflows as judged by end users

Answer: 1→A, 2→B

Q23. Order the ALM promotion steps (1 = first, 4 = last) Items: Develop in dev environment | Export as managed solution | Import/test in test environment | Deploy to production environment

Answer: 1) Develop in dev environment → 2) Export as managed solution → 3) Import/test in test environment → 4) Deploy to production environment

Q24. Match the Responsible AI principle to its implementation example Items: 1) Fairness 2) Privacy & Security 3) Accountability Targets: A) Encrypting data at rest and enforcing least-privilege connections B) Testing the agent across diverse user groups to avoid biased outcomes C) Requiring human approval before high-impact autonomous actions

Answer: 1→B, 2→A, 3→C

Q25. Match the knowledge source type to the best-fit scenario Items: 1) Dataverse table 2) Azure AI Search index 3) Public website Targets: A) Structured CRM records needing real-time query B) Heterogeneous multi-source content needing custom relevance tuning C) Publicly available product documentation pages

Answer: 1→A, 2→B, 3→C

Q26. Order the incident response steps for an agent producing harmful output (1 = first, 4 = last) Items: Disable/pause the agent | Root-cause analysis using telemetry/logs | Notify stakeholders | Apply fix and re-test before re-enabling

Answer: 1) Disable/pause the agent → 2) Notify stakeholders → 3) Root-cause analysis using telemetry/logs → 4) Apply fix and re-test before re-enabling

Q27. Match the orchestration pattern to its description Items: 1) Hub-and-spoke (router + specialists) 2) Sequential pipeline 3) Peer-to-peer via A2A Targets: A) One agent classifies and hands off to the right specialist B) Agents pass output as input to the next in fixed order C) Independent agents negotiate/delegate tasks directly to each other

Answer: 1→A, 2→B, 3→C

Q28. Match the monitoring signal to what it reveals Items: 1) Token usage trend 2) Escalation rate 3) Latency P95 Targets: A) Cost growth and potential prompt-size issues B) Agent capability gaps needing more training/topics C) User experience/responsiveness under load

Answer: 1→A, 2→B, 3→C

Q29. Order the steps to add a new knowledge source to a production agent safely Items: Validate source content quality/format | Test in a non-production environment | Add source in dev and configure grounding | Promote via managed solution to production

Answer: 1) Validate source content quality/format → 2) Add source in dev and configure grounding → 3) Test in a non-production environment → 4) Promote via managed solution to production

Q30. Match the governance control to the risk it addresses Items: 1) DLP policy 2) Environment strategy (dev/test/prod separation) 3) Content moderation/filters Targets: A) Sensitive data flowing to unapproved connectors B) Untested changes reaching real users C) Generation of harmful or inappropriate content

Answer: 1→A, 2→B, 3→C

Section 4 — Drop-Down Selection (Complete the Diagram/Statement) (10)

Fill each blank by choosing from the options in brackets.

Q31. A finance approval agent must never approve payments over policy limits without a human. This requires [Human-in-the-loop checkpoint / Fully autonomous execution / Generative-only response].

Answer: Human-in-the-loop checkpoint

Q32. To search across PDFs, database records, and API results as a single grounded knowledge base, use [Azure AI Search / A single Dataverse table / Plain text topic responses].

Answer: Azure AI Search

Q33. For repeatable, environment-isolated deployment of Copilot Studio agents, use [Managed solutions promoted through environments / Manual copy-paste of topics / Shared dev/prod environment].

Answer: Managed solutions promoted through environments

Q34. To let one agent call a specialized pricing-calculation agent hosted elsewhere, the recommended open protocol is [Agent2Agent (A2A) / SMTP / FTP].

Answer: Agent2Agent (A2A)

Q35. To give an agent access to a specific external tool's live data (e.g., a ticketing system) in a standardized way, use [Model Context Protocol (MCP) / RSS feed / Static CSV upload].

Answer: Model Context Protocol (MCP)

Q36. To detect and block attempts to override an agent's system instructions via user or document content, enable [Prompt shields / Higher temperature / Larger context window].

Answer: Prompt shields

Q37. To track cost, latency, and failure trends for agents running in Foundry over time, integrate with [Azure Monitor / Outlook Calendar / SharePoint versioning].

Answer: Azure Monitor

Q38. To ensure sensitive customer data isn't sent to an ungoverned third-party connector, configure [Data Loss Prevention (DLP) policies / A longer system prompt / A new topic].

Answer: Data Loss Prevention (DLP) policies

Q39. For a scenario needing flexible handling of open-ended, unpredictable user requests without enumerating every intent, choose [Generative orchestration / Classic topics with fixed trigger phrases / No AI].

Answer: Generative orchestration

Q40. To validate that an agent solution passes governance/quality checks automatically before deployment, include a [Solution checker in the CI/CD pipeline / Manual email approval only / No validation step]. **A**

Answer: Solution checker in the CI/CD pipeline

Section 5 — Case Study (Multi-Part, Scenario-Based)

Case Study: Contoso Manufacturing

Background: Contoso Manufacturing runs Dynamics 365 Finance, Dynamics 365 Supply Chain Management, and Dynamics 365 Sales. They want to deploy:

- A Copilot Studio customer-facing agent for order status and returns.
- A Foundry-hosted internal agent that analyzes supply chain disruptions using data from SQL, flat files, and a public logistics API.
- A multi-agent setup where the customer-facing agent can delegate complex return-eligibility questions to an internal policy agent.

Requirements:

- Customer data must never leave approved, DLP-governed connectors.
- The supply chain agent's answers must be traceable to source data.
- Deployment must be fully automated with dev/test/prod isolation.
- High-impact actions (e.g., approving returns over \$5,000) require human approval.

- The company must be able to monitor cost and latency across all agents.

Q41. Which knowledge source strategy should you recommend for the supply chain disruption agent, given its mixed SQL, flat-file, and API data?

- A. Azure AI Search index built from all three sources
- B. Only a Dataverse table
- C. Only the public API, ignoring internal data
- D. No structured knowledge source, generative-only

Answer: A — heterogeneous sources need a unified, custom index.

Q42. What governance control most directly ensures customer data doesn't flow through unapproved connectors?

- A. DLP policy scoping connectors into business/non-business groups
- B. A longer topic name
- C. Disabling the agent's avatar
- D. Increasing max tokens

Answer: A

Q43. How should the return-eligibility delegation between the customer-facing agent and internal policy agent be implemented?

- A. Agent2Agent (A2A) communication pattern
- B. Copy-pasting the policy document into every customer chat
- C. A shared Excel file updated manually
- D. Disabling the internal agent

Answer: A

Q44. What should trigger human review for a return request?

- A. Return value exceeding the defined policy threshold (e.g., \$5,000)
- B. Any return under \$10
- C. Every single interaction regardless of value
- D. Never — full autonomy for all returns

Answer: A

Q45. Which deployment approach satisfies "fully automated with dev/test/prod isolation"?

- A. CI/CD pipeline promoting managed solutions across managed environments
- B. Manual export/import by a single admin
- C. Editing directly in production
- D. Ad hoc publishing from developer sandboxes

Answer: A

Q46. What should you **configure to make the supply chain agent's answers traceable to source data?**

- A. Grounding with citations enabled against the AI Search index
- B. Disabling citations for a cleaner UI
- C. Free-form generative answers with no source linkage
- D. Removing the knowledge source

Answer: A

Q47. What is the best way to monitor cost and latency across all three agents centrally?

- A. Azure Monitor/Application Insights dashboards spanning the agents' hosting environments
- B. Manually asking each agent for its own stats
- C. Reviewing Dynamics 365 Sales pipeline reports
- D. No monitoring needed post-launch

Answer: A

Q48. Which testing type validates that the return-eligibility handoff between the two agents functions **correctly end-to-end?**

- A. Integration testing
- B. Unit testing of a single topic in isolation
- C. Load testing of the UI theme
- D. Spelling/grammar review only

Answer: A

Section 6 — Yes/No Hotspot Questions

(Format note: in the real exam, these appear as a set of statements under one scenario, answered Yes/No, and cannot be revisited after submission — so treat each set as final once you decide.)

Scenario: A sales team uses a Copilot Studio agent via headset in a call center. The agent can look up order status, and can also issue refunds. Refunds under \$500 are auto-approved; refunds \$500+ require supervisor approval. All conversations are logged for quality review, but credit card numbers must never appear in logs.

Statement 1: The agent should escalate to a human supervisor automatically when a refund request is \$500 or more.

Answer: Yes —

Reason: this matches the stated business rule for human-in-the-loop approval above a threshold; it's a deterministic policy, not a judgment call for the agent to override.

Statement 2: Credit card numbers should be redacted or excluded from conversation logs even though full logging is otherwise required.

Answer: Yes —

Reason: sensitive payment data (PCI-scoped) must be excluded/masked from logs regardless of a general logging requirement; this is a security/compliance control, not optional.

Statement 3: The agent should be allowed to auto-approve a \$500 refund if the customer sounds frustrated, to improve satisfaction.

Answer: No —

Reason: the \$500 threshold is a fixed governance rule; allowing sentiment to override a compliance threshold undermines the control and introduces inconsistent, unaudited exceptions.

Scenario 2: An HR-facing autonomous agent in Dynamics 365 can update employee records and trigger onboarding workflows without a human in the loop, based on a nightly batch trigger.

Statement 1: Because the agent runs autonomously, it does not need any monitoring or telemetry review.

Answer: No —

Reason: autonomous agents require monitoring even more than human-in-the-loop ones, since there's no real-time human check on individual actions; telemetry is how errors get caught.

Statement 2: Changes made by the autonomous agent should still be traceable to a specific run/trigger for audit purposes.

Answer: Yes —

Reason: auditability of autonomous actions is a core accountability requirement, especially for HR/PII data changes.

Statement 3: The agent should have unrestricted write access to all HR fields, including compensation, to simplify configuration.

Answer: No —

Reason: least-privilege access design means the agent's permissions should be scoped tightly to onboarding-relevant fields, not broadened for convenience.

How to Use This Bank

- Work through each section untimed first, writing out *why* each wrong answer is wrong — that reasoning is what the real exam tests (per your notes, it's scenario/trade-off based, not recall-based).
- Re-take Section 5 (case study) timed, since real case studies are the format most people underestimate.
- Since Deploy-domain content (ALM, monitoring, security, Responsible AI) is ~40-45% of the real exam, most of Sections 1-6 here lean that direction on purpose — if you found those items easy, that's a good sign; if not, that's where to spend remaining study time.
- This set is deliberately harder than the first — nothing is a clean "obviously right" answer. The pattern to watch for across all of it: the correct choice almost always places a control at the *right layer* (workflow gate vs. prompt instruction vs. governance policy vs. monitoring), rather than just picking the most restrictive or most "trust the AI" option.
- Two case studies (Northwind Global Logistics — multi-region supply chain, and Fabrikam Financial Services — regulated finance) each carry compounding constraints across questions, the way real AB-100 case studies do, so getting Q21 wrong will cascade into Q24.
- If you want, I can also build a timed mock exam that mixes both banks (this one + the earlier easier set) into a single 40-question run to simulate the real 120-minute pacing.

AB-100 Practice Bank — Expert/Architect Level (Difficulty 9/10)

These go deeper than the first set: multi-constraint scenarios, competing trade-offs, and "least-wrong" decisions rather than obviously-correct ones — the way real AB-100 case studies work at the architect level. Each answer includes the trade-off reasoning, including why the tempting alternative is wrong.

Section 1 — Drag-and-Drop (10) — Architect Level

Q1. Match each cross-platform integration requirement to the correct protocol/mechanism. Items:

1. A Copilot Studio agent must delegate a claims-fraud-scoring task to a specialized Foundry agent owned by a different team, with each side able to negotiate capability and reject the task if overloaded.
2. A Foundry agent needs live, standardized access to a third-party warehouse management system's inventory API, where the API provider publishes a standard tool schema.
3. Two Dynamics 365 environments (Sales and Finance) need near-real-time record sync as part of business logic, not agent reasoning.
4. A single agent needs to expose its own capabilities so other agents in the enterprise can discover and call it without custom point-to-point integration.

Targets: A) Agent2Agent (A2A) protocol B) Model Context Protocol (MCP) C) Dataverse virtual tables / Power Platform data integration (non-agentic) D) A2A agent card / capability discovery

Answer: 1→A, 2→B, 3→C, 4→D

Why it's hard: 1 and 4 both "sound like A2A." The distinguishing detail is *negotiation between peer agents mid-task* (A2A protocol proper) vs. *publishing discoverable capabilities* (the agent card/discovery mechanism within A2A) — architects need to separate the transport/negotiation layer from the discovery layer. Item 3 is a trap: it mentions two Dynamics apps and sounds "agentic," but it's plain data integration, not agent orchestration — using A2A here would be over-engineering.

Q2. Match the failure mode to the correct architectural mitigation (not the generic one). Items:

1. An autonomous agent's nightly batch run silently fails for 3 days before anyone notices, causing a compliance gap.
2. A multi-agent pipeline works in test but degrades in production because the router agent's classification confidence drops under real-world phrasing variance.
3. A knowledge-grounded agent gives correct-sounding but outdated pricing because the source document was updated but the index wasn't refreshed.
4. An agent with tool-calling access is tricked via a crafted support ticket into exfiltrating a customer's PII to an external webhook tool.

Targets: A) Scheduled index/knowledge-source refresh with staleness alerting, not just initial grounding setup B) Active monitoring with automated alerting on run failure/anomaly, not just log retention C) Tool allow-listing plus output-side content inspection before external tool execution, not just prompt shields alone D) Confidence-threshold fallback to human/topic-based routing plus continuous evaluation against production-representative data, not just pre-launch testing

Answer: 1→B, 2→D, 3→A, 4→C

Why it's hard: Every item has an "almost right" generic answer (e.g., "just add prompt shields" for #4, "just retrain the model" for #2). The correct answers require layered controls: monitoring ≠ logging, prompt shields alone don't stop *output-side* exfiltration via legitimate-looking tool calls, and grounding is not a one-time setup — it needs a refresh lifecycle.

Q3. Order the steps for safely rolling out a change to a production multi-agent solution that spans Copilot Studio and Foundry, where the two platforms have separate ALM pipelines. Items: Coordinate a joint release window and rollback plan across both pipelines | Deploy Foundry-side change to its test environment and run agent evaluations | Deploy Copilot Studio-side change to its test environment and validate against the already-tested Foundry endpoint | Promote both to production within the agreed window, verifying cross-platform handoffs post-deploy | Independently unit-test each platform's change in its own dev environment

Answer:

1. Independently unit-test each platform's change in its own dev environment
2. Deploy Foundry-side change to its test environment and run agent evaluations
3. Deploy Copilot Studio-side change to its test environment and validate against the already-tested Foundry endpoint

4. Coordinate a joint release window and rollback plan across both pipelines
5. Promote both to production within the agreed window, verifying cross-platform handoffs post-deploy

Why it's hard: Two independent ALM pipelines must be sequenced (Foundry validated before Copilot Studio tests against it, since Copilot Studio depends on the Foundry contract) and then *re-synchronized* for a coordinated release — architects must reason about pipeline dependency order, not just "test then deploy" in isolation.

Q4. Match the enterprise governance escalation path to the correct control layer.

Items:

1. A business unit wants to allow their agent to connect to a personal Gmail account as a data source.
2. An agent needs elevated Dataverse permissions beyond its current security role to complete a new use case.
3. A newly onboarded agent must be reviewed for content safety and bias before general availability.
4. An agent's total monthly Azure OpenAI token spend is trending 300% over budget.

Targets: A) DLP policy governance (tenant/environment-level connector classification)
B) Dataverse security role and column-level security review
C) Responsible AI review gate as part of the release/ALM checklist
D) FinOps/cost governance with budget alerts and quota management

Answer: 1→A, 2→B, 3→C, 4→D

Why it's hard: All four are "governance," but architect-level competence means routing each to the *specific* control plane responsible — mixing these up (e.g., treating a cost overrun as a DLP issue) is the kind of wrong-layer answer the exam penalizes.

Q5. Order the steps to design a Responsible AI-compliant autonomous agent that approves supplier payments up to a variable, currency-dependent threshold.

Items: Implement human-approval gate for any transaction exceeding the resolved threshold | Log every decision with the resolved threshold and rationale for audit | Externalize the threshold logic into configurable business rules (not hardcoded in the prompt) | Define the threshold policy per currency/region with legal and finance stakeholders | Run adversarial testing to confirm the agent cannot be prompted into bypassing the threshold check

Answer:

1. Define the threshold policy per currency/region with legal and finance stakeholders
2. Externalize the threshold logic into configurable business rules (not hardcoded in the prompt)
3. Implement human-approval gate for any transaction exceeding the resolved threshold
4. Run adversarial testing to confirm the agent cannot be prompted into bypassing the threshold check
5. Log every decision with the resolved threshold and rationale for audit

Why it's hard: The natural instinct is to jump straight to "add a human approval step." The architect-level ordering requires policy definition and rule externalization *before* the control is built (so the control enforces the right thing), then adversarial validation *before* trusting it in production, with audit logging as the closing, continuous layer — not a one-time step.

Q6. Match the multi-agent orchestration anti-pattern to its consequence. Items:

1. Every specialist agent shares one enormous system prompt covering all domains "just in case."
2. The router agent has no fallback path when no specialist's confidence exceeds threshold.
3. Specialist agents write directly to a shared Dataverse table with no ownership boundaries.
4. Agent-to-agent calls have no timeout or circuit breaker.

Targets: A) A single struggling downstream agent can cascade latency/failures through the whole pipeline B) Data integrity conflicts and unclear accountability for record changes C) Prompt bloat causing higher cost, latency, and diluted instruction-following D) User requests silently fail or produce a generic non-answer with no recovery path

Answer: 1→C, 2→D, 3→B, 4→A

Q7. Match the data residency/sovereignty requirement to the correct architectural response. Items:

1. EU customer data must not be processed by a model hosted outside the EU.

2. A regulator requires that all agent-generated financial recommendations retain full input/output logs for 7 years.
3. A conglomerate wants one Foundry hub reused across regional subsidiaries but with regional data isolation.
4. A healthcare customer requires PHI to never be sent to a knowledge source without a signed BAA-equivalent agreement in place.

Targets: A) Deploy the Foundry model/agent in an EU region and pin data processing/storage there B) Long-term log retention and immutable audit storage policy, exceeding default retention C) Per-region projects/resource groups with isolated storage and access boundaries under the shared hub D) Restrict knowledge source connectors to only compliant, contractually-covered data stores

Answer: 1→A, 2→B, 3→C, 4→D

Q8. Order the steps to migrate a legacy rule-based Power Virtual Agents (classic) bot into a modern generative-orchestration Copilot Studio agent without breaking existing customer-facing SLAs. Items: Run the new agent in shadow mode against live traffic, comparing outputs without serving them | Cut over a small traffic percentage behind a feature flag, monitoring escalation/error rates | Inventory and categorize existing topics by criticality and complexity | Rebuild high-criticality topics as still-deterministic classic topics, and low-criticality/high-variance ones as generative | Fully cut over and decommission the legacy bot once metrics hold steady

Answer:

1. Inventory and categorize existing topics by criticality and complexity
2. Rebuild high-criticality topics as still-deterministic classic topics, and low-criticality/high-variance ones as generative
3. Run the new agent in shadow mode against live traffic, comparing outputs without serving them
4. Cut over a small traffic percentage behind a feature flag, monitoring escalation/error rates
5. Fully cut over and decommission the legacy bot once metrics hold steady

Why it's hard: The trap is assuming "generative orchestration" should replace everything. The architect answer is a *hybrid* migration that keeps deterministic topics deterministic where SLA/compliance criticality demands it, with a staged, measurable cutover — not a wholesale rip-and-replace.

Q9. Match the telemetry anomaly to the most likely root cause an architect should investigate first. Items:

1. Token usage per conversation doubled after a knowledge source update, with no change to user behavior.
2. Escalation rate spiked only for one specific intent category.
3. P95 latency degraded tenant-wide after adding one new external tool integration.
4. Cost per resolved conversation improved, but customer satisfaction score dropped.

Targets: A) The new knowledge source is returning excessively large or poorly-chunked context into the prompt B) A recent change to that intent's topic/prompt reduced answer quality or added ambiguity C) The new tool call is synchronous and blocking, or lacks a timeout, dragging overall response time D) Cost optimization (e.g., shorter responses, cheaper model) is being over-applied at the expense of answer quality

Answer: 1→A, 2→B, 3→C, 4→D

Q10. Order the decision steps for choosing between "extend an existing Dynamics 365 Copilot" vs. "build a net-new custom agent" for a new customer-service AI capability. Items: Check whether existing Copilot extensibility (knowledge sources, plugins) can meet 80%+ of requirements | Assess if the required capability needs cross-app orchestration beyond one Dynamics app's scope | If extensibility suffices, extend the built-in Copilot instead of building new | If cross-app or highly custom logic is required, design a custom Copilot Studio/Foundry agent that integrates with, rather than replaces, the built-in Copilot | Document the decision rationale for future architects/auditors

Answer:

1. Check whether existing Copilot extensibility (knowledge sources, plugins) can meet 80%+ of requirements
2. Assess if the required capability needs cross-app orchestration beyond one Dynamics app's scope
3. If extensibility suffices, extend the built-in Copilot instead of building new
4. If cross-app or highly custom logic is required, design a custom Copilot Studio/Foundry agent that integrates with, rather than replaces, the built-in Copilot

5. Document the decision rationale for future architects/auditors

Section 2 — Drop-Down / Complete-the-Architecture (10) — Architect Level

Q11. A global enterprise needs one Foundry hub shared across five regional business units, each requiring isolated model access, cost tracking, and content policies, but sharing a common enterprise-approved model catalog. The correct architecture is: one hub with [per-region Foundry projects inheriting the shared model catalog but with isolated resources and policies / five fully separate Foundry resources with no shared catalog / a single project shared by all regions with role-based visibility only].

Answer: per-region Foundry projects inheriting the shared model catalog but with isolated resources and policies

Why the distractors fail: Fully separate resources (option 2) loses the governance benefit of a shared, vetted model catalog; a single shared project (option 3) fails the isolation requirement outright — RBAC visibility isn't the same as resource/cost/policy isolation.

Q12. An agent must reason over 200GB of continuously updating operational data where sub-second freshness matters for some queries but not others. The architecture should use [a tiered approach: real-time API/tool calls for time-sensitive data, and a periodically-refreshed AI Search index for the rest / a single AI Search index refreshed nightly for everything / real-time API calls for all queries regardless of data type].

Answer: a tiered approach: real-time API/tool calls for time-sensitive data, and a periodically-refreshed AI Search index for the rest

Why the distractors fail: An all-nightly-index approach breaks freshness for time-sensitive queries; an all-real-time approach is unnecessary cost/latency overhead for data that doesn't need it.

Q13. A regulated agent's output must be reproducible for audit — given the same input and same knowledge state, it should produce a consistent, explainable answer. You should configure [low temperature, strict grounding, and versioned/pinned knowledge source snapshots / high temperature for more natural responses / grounding disabled to allow flexible phrasing].

Answer: low temperature, strict grounding, and versioned/pinned knowledge source snapshots

Q14. A multi-agent solution needs the router agent to hand off full conversational context (not just the last message) to a specialist agent so the specialist doesn't ask the user to repeat themselves. This requires [a context-passing handoff mechanism (e.g., shared conversation state/session object across the A2A call) / restarting the

conversation with the specialist agent from scratch / summarizing only the agent name to the specialist].

Answer: a context-passing handoff mechanism (e.g., shared conversation state/session object across the A2A call)

Q15. A CISO requires that no agent tool call can silently exfiltrate data to an arbitrary external endpoint, even if a malicious prompt tries to instruct it to do so. The control to implement is [an egress allow-list restricting tool/connector destinations to pre-approved endpoints / relying on the model's own judgment / disabling all tools, breaking required functionality].

Answer: an egress allow-list restricting tool/connector destinations to pre-approved endpoints

Q16. An architect must decide the ALM branching strategy for a Copilot Studio + Foundry solution with three parallel feature teams. The recommended approach is [feature branches per team, merged via pull request into a shared main/trunk, with automated solution-checker validation on merge / one shared branch with no isolation / each team maintaining a fully separate, never-merged fork].

Answer: feature branches per team, merged via pull request into a shared main/trunk, with automated solution-checker validation on merge

Q17. An agent handling healthcare intake must ensure PHI never appears in application logs used for general debugging, while still supporting compliance audit trails. The design should separate [a redacted/masked debug log stream from a separately access-controlled, encrypted audit log stream retaining full detail / one single log stream with PHI included / no logging at all to avoid risk].

Answer: a redacted/masked debug log stream from a separately access-controlled, encrypted audit log stream retaining full detail

Q18. A cost-conscious enterprise wants to cap runaway spend from an autonomous agent that could, in a failure loop, call a paid API repeatedly. The mitigation is [rate limiting/circuit breakers plus budget alerts and hard quota caps at the resource level / trusting the agent to self-limit via prompt instructions only / removing monitoring to reduce overhead].

Answer: rate limiting/circuit breakers plus budget alerts and hard quota caps at the resource level

Q19. A company wants a single pane of glass showing agent health (errors, latency, escalations) across Copilot Studio, Foundry, and Dynamics 365 Copilot simultaneously. The recommended architecture is [centralized telemetry export from each platform into

Azure Monitor/Log Analytics with unified dashboards / separate dashboards per platform reviewed manually / no centralization, relying on each team's own tools].

Answer: centralized telemetry export from each platform into Azure Monitor/Log Analytics with unified dashboards

Q20. An agent solution must support instant rollback if a new generative orchestration model version degrades quality in production. The architecture should include [blue/green or canary deployment with the ability to route back to the prior model version / direct in-place upgrade with no rollback path / manual redeployment from scratch if issues arise].

Answer: blue/green or canary deployment with the ability to route back to the prior model version

Section 3 — Case Studies (8 questions across 2 case studies) — Architect Level

Case Study A: Northwind Global Logistics (4 questions)

Background: Northwind operates across EU, US, and APAC. They run Dynamics 365 Supply Chain Management (all regions) and Dynamics 365 Sales (US and EU only). They want a multi-agent solution:

- A "Dispatch" agent (Copilot Studio) that customer-facing dispatch staff use to check shipment status and re-route shipments.
- A "Risk" agent (Foundry) that predicts shipment delay risk using real-time carrier APIs, weather data, and historical Dataverse records, and can autonomously re-route low-risk, low-value shipments (<\$1,000) without approval.
- Re-routes over \$1,000 or involving hazardous materials require supervisor approval regardless of predicted risk.
- EU shipment data must stay in EU-hosted resources.
- Executives want a single dashboard of agent cost, latency, and override/escalation rates across both agents and all regions.

Q21. How should the Risk agent's autonomous re-routing threshold logic be implemented?

- A. Hardcoded into the agent's system prompt as a numeric rule
- B. As externalized, configurable business rules evaluated before the agent acts, combining value threshold AND hazardous-material flag (not value alone)
- C. Left entirely to the model's judgment based on training data

D. Applied only to EU shipments

Answer: B — Two independent conditions (value AND hazmat) must both gate autonomy; a prompt-only or single-condition rule under-implements the stated policy, and hardcoding in the prompt makes it unauditable and hard to update.

Q22. How should EU data residency be architected given the Risk agent needs cross-region historical data for prediction accuracy?

A. Host EU shipment data and processing in EU Foundry projects/resources, and use aggregated/anonymized cross-region signals (not raw EU records) for the global model where needed

B. Move all data to a single US region for simplicity

C. Ignore residency since it's an internal analytics use case

D. Store EU data in EU but process it in US Foundry resources

Answer: A — Residency requires both storage *and* processing to stay in-region for EU records; option D violates that by processing EU data outside the EU even if storage is compliant.

Q23. What is the correct architecture for the unified executive dashboard across two platforms and three regions?

A. Centralized telemetry aggregation (e.g., Azure Monitor/Log Analytics) pulling from each region's and platform's resources into unified, role-scoped dashboards

B. Three separate regional dashboards emailed weekly

C. A single Dynamics 365 Sales report, since Sales already has dashboards

D. Manual quarterly spreadsheet compilation

Answer: A

Q24. A hazmat shipment valued at \$600 is flagged low-value by the Risk agent's value rule. What should happen?

A. The agent auto-reroutes it since it's under \$1,000

B. The agent escalates for supervisor approval because the hazmat flag independently requires approval regardless of value

C. The agent asks the customer directly

D. The agent ignores the hazmat flag since value is the primary rule

Answer: B — Tests whether the candidate correctly implemented Q21's AND logic; a single-condition implementation would wrongly auto-approve this.

Case Study B: Fabrikam Financial Services (4 questions)

Background: Fabrikam is a regulated financial services firm using Dynamics 365 Finance and Copilot Studio. They're deploying:

- A customer-facing "Advisor Assistant" agent that answers general account questions and can initiate (not complete) fund transfers.
- Transfers require a separate, human-staffed verification step before execution — never autonomous completion.
- Regulators require full conversation transcripts retained for 7 years, with PII masked in any transcript used for internal QA/training purposes.
- The firm was previously fined for a chatbot giving unlicensed financial advice, so the new agent must strictly avoid generating personalized investment recommendations.
- The firm wants to reuse the Advisor Assistant's transfer-initiation capability inside a separate internal "Ops" Foundry agent used by staff.
-

Q25. How should the "never autonomous completion" requirement for transfers be enforced architecturally, not just via prompt instruction?

A. A hard workflow gate outside the agent's own reasoning (e.g., a separate approval/execution step in the business process) that the agent cannot bypass regardless of what it decides

B. A system prompt instruction telling the agent never to complete transfers

C. Trusting the agent's generative reasoning to refuse

D. Rate-limiting transfer requests

Answer: A — Compliance-critical controls must live outside model reasoning, in the workflow/process layer, because prompt instructions (B/C) are not a reliable enforcement boundary — this is the core "architectural vs. prompt-level control" distinction the exam tests repeatedly.

Q26. Given the prior regulatory fine, how should the agent's knowledge sources and orchestration be constrained to avoid generating personalized investment advice?

- A. Restrict knowledge sources to general product/account information only, and use classic topics or tightly scoped generative orchestration with explicit refusal patterns for advice-seeking questions, routing those to a licensed human advisor
- B. Give the agent full generative orchestration with access to all customer portfolio data for "helpfulness"
- C. Add a disclaimer but otherwise allow open-ended advice generation
- D. Remove the agent entirely and use only human advisors

Answer: A — D over-corrects and eliminates the business value entirely; B recreates the exact risk that caused the fine; a disclaimer (C) doesn't change the underlying generation risk. A is the balanced architectural control.

Q27. How should PII masking for the 7-year transcript retention requirement be designed?

- A. One retention system with two access paths: full-detail transcripts under strict access control for regulatory audit, and a masked/redacted derivative for internal QA/training use
- B. Mask PII in all copies, including the regulatory audit copy
- C. Keep only masked transcripts to reduce risk
- D. Keep full-detail transcripts everywhere since 7-year retention is required

Answer: A — B and D each satisfy only one of the two competing requirements (audit needs full detail; QA/training use needs masking) — the architect answer is a dual-path design satisfying both simultaneously.

Q28. What is the correct way to reuse the transfer-initiation capability inside the internal Ops Foundry agent?

- A. Expose transfer-initiation as a shared, versioned tool/capability (e.g., via MCP or a shared API) that both agents call, rather than duplicating the logic in each agent
- B. Copy and paste the transfer-initiation prompt logic into the Ops agent
- C. Give the Ops agent direct database write access instead
- D. Build the capability twice, independently, for each agent

Answer: A — B and D create duplicated, divergent logic that will drift out of sync and multiply the compliance surface area; C bypasses the governed workflow gate established in Q25 entirely.

Section 4 — Yes/No Hotspot (Architect Level)

Scenario 1

A global insurance company's Foundry-hosted claims-triage agent uses generative orchestration to summarize claims and recommend a routing queue (fast-track, standard, investigation-required).

It has read access to claims data and write access only to a "recommended queue" field — never to the actual payout decision. The agent flags claims for investigation when the model detects potential inconsistencies. All investigation-flagged claims are reviewed by a human within 24 hours, but non-flagged claims route automatically with no human touch.

Statement 1: Because the agent never touches the actual payout decision, it doesn't need adversarial testing against attempts to manipulate its routing recommendation.

Answer: No —

Reason: routing itself is a consequential decision (fast-tracking a fraudulent claim, or delaying a legitimate one via investigation, harms a real customer), so manipulation of the routing recommendation is still a material risk requiring adversarial testing, even though the agent doesn't directly authorize payment.

Statement 2: The 24-hour human review SLA for investigation-flagged claims should be monitored and alerted on as part of the agent's operational telemetry.

Answer: Yes —

Reason: an SLA tied to a compliance/customer-outcome commitment is exactly the kind of signal that needs active monitoring, not just logging, since a missed SLA is itself an incident.

Statement 3: Non-flagged claims routing automatically with no human touch is acceptable given the write access is scoped only to the queue field, not the payout.

Answer: Yes —

Reason: this is a case where the least-privilege write scope (queue field only, not payout) combined with downstream human-owned payout decisions is a legitimate, already-safe design — not every automated path requires a human gate; the architect skill being tested here is recognizing when a control is sufficient, not just adding more gates reflexively.

Scenario 2

A retail bank's Copilot Studio "Onboarding" agent helps new customers open accounts. It's built with **generative orchestration**, has knowledge sources containing product terms, and can call a backend API to actually create the account record once the customer confirms all details. The bank recently found that **under specific phrasing, the agent could be led to create an account with a product type not actually offered in the customer's home state (a compliance violation, since not all products are licensed in all states).**

Statement 1: Adding a post-generation validation step that **checks the requested product against the customer's state license table before the account-creation API call fires** would prevent this specific failure mode.

Answer: Yes —

Reason: this is a deterministic, **rule-based check happening outside the model's generative reasoning, at the exact point of consequential action (before the API call)** — the correct architectural placement for this class of control, and it directly addresses the described failure regardless of how the model was talked into the wrong state.

Statement 2: Since the root cause was a **prompt-phrasing manipulation**, the only necessary fix is to update the system prompt to **explicitly forbid unlicensed products**.

Answer: No —

Reason: prompt-only fixes are **not reliable enforcement boundaries against adversarial phrasing** (the same class of gap that caused the bug); a deterministic validation gate outside the model, as in Statement 1, **is required as the durable fix** — this reinforces the theme that consequential actions need **process/workflow controls**, not just instruction tuning.

Statement 3: This incident indicates the account-creation API call should never have been given directly to **a generative-orchestration agent**, and the entire onboarding flow should be rebuilt as **classic topics only**.

Answer: No —

Reason: this over-corrects; the actual gap was a missing validation **gate at the action boundary**, not the presence of **generative orchestration** itself — generative orchestration is well-suited to **the conversational/data-gathering part of onboarding**, and the fix (Statement 1) resolves the issue without **discarding the appropriate architecture for the rest of the flow**.
